Лабораторна робота 14

Тема: Поєднання веб-додатку та машинного навчання для класифікації текстових даних

Мета: Освоєння процесу поєднання моделі машинного навчання з веб-додатком, набуття навичок роботи з текстовими даними, навчання та збереження моделі класифікації, а також створення простого користувацького інтерфейсу для взаємодії з навченою моделлю.

Хід роботи

Перед початком програмування необхідно зрозуміти загальну логіку роботи створюваної системи. У межах даної лабораторної роботи реалізація побудована таким чином, щоб окремо виконувати навчання моделі та її подальше використання у веб-додатку.

На першому етапі виконується підготовка та аналіз текстових даних, після чого здійснюється навчання моделі машинного навчання. Цей етап виконується лише один раз, оскільки його результатом є готова навчена модель та векторизатор, які зберігаються у вигляді окремих файлів. Надалі ці файли використовуються у веб-додатку без необхідності повторного навчання моделі.

Другий етап полягає у використанні навченої моделі у веб-додатку. Користувач вводить текст через веб-інтерфейс, після чого введений текст автоматично передається у модель машинного навчання. Модель аналізує отримане повідомлення та повертає результат класифікації, який відображається користувачу у браузері у зрозумілому вигляді.

Для демонстрації поєднання методів машинного навчання з веб-додатком було обрано ***задачу класифікації електронних листів***.

Суть цієї задачі полягає у визначенні, чи є конкретне текстове повідомлення небажаним (**SPAM**), або ж належить до звичайних електронних листів (**NOT SPAM**). Оскільки кожне повідомлення може належати лише до одного з двох можливих класів, дана задача відноситься до задач бінарної класифікації.

Для навчання моделі було використано датасет, завантажений з платформи Kaggle.

Після завантаження датасет було розархівовано та збережено у папці проєкту у вигляді файлу ***emails.csv***.

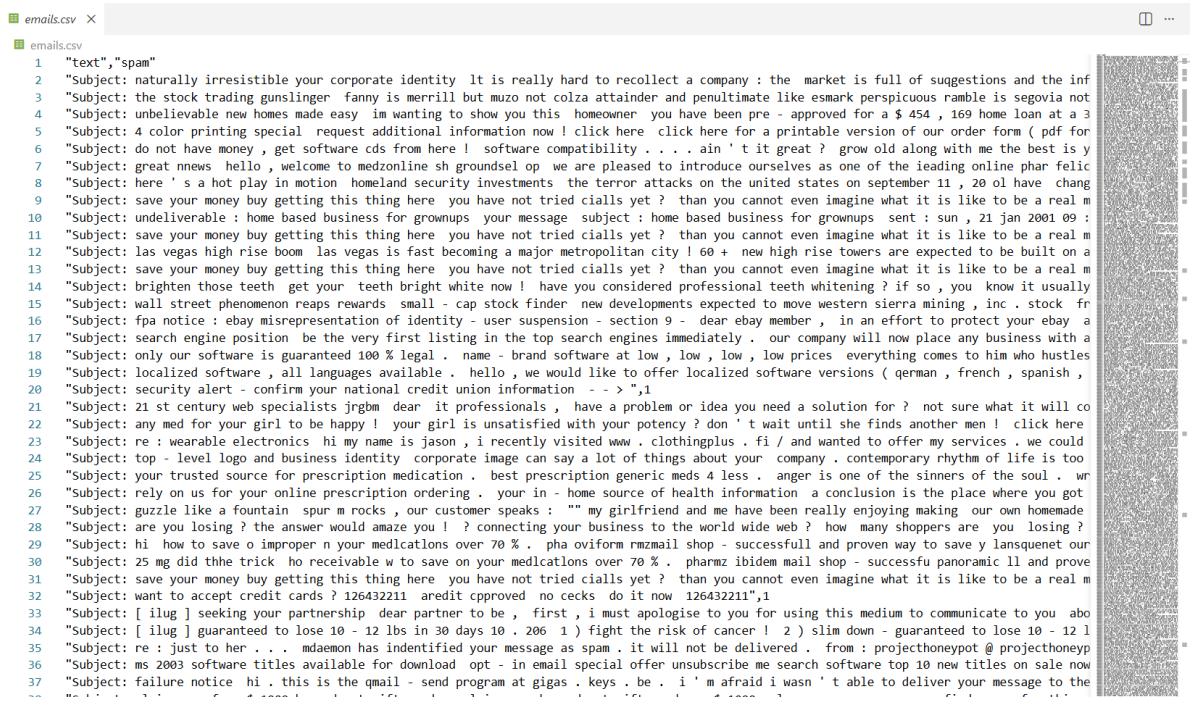


Рисунок 1 - Файл emails.csv

Файл містить два стовпці:

1. ***text*** (повний текст електронного листа)
2. ***spam*** (числова мітка класу, що визначає тип повідомлення. 1 – спам-повідомленням, тоді як значення 0 використовується для позначення звичайних, не спам-листів.)

Наявність попередньо розмічених даних є необхідною умовою для навчання моделі, оскільки саме ці мітки використовуються для формування правил класифікації нових повідомлень.

Після підготовки датасету та визначення задачі було створено проєкт, структура якого наведена на рисунку 2.

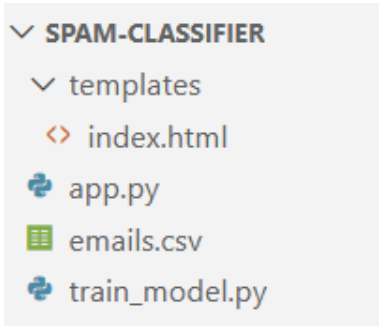


Рисунок 2 — Структура проєкту

Папка проєкту містить такі основні елементи:

- папку ***templates***, у якій зберігається HTML-шаблон веб-сторінки;
- файл ***app.py***, що відповідає за роботу веб-додатку та взаємодію з користувачем;

- файл ***emails.csv***, який містить датасет для навчання моделі;
- файл ***train_model.py***, у якому реалізовано процес підготовки даних та навчання моделі машинного навчання.

Така організація файлів спрощує розуміння проєкту та дозволяє легко відокремити етап навчання моделі від етапу її використання у веб-додатку. Далі детально та покроково розглянемо кожен із цих етапів.

Навчання моделі машинного навчання

Процес навчання моделі машинного навчання реалізовано у файлі ***train_model.py***. Саме в цьому файлі відбувається завантаження датасету, підготовка текстових даних, навчання моделі класифікації та збереження отриманих результатів для подальшого використання у веб-додатку.

На початку файлу підключаються необхідні бібліотеки:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
import pickle
import re
```

Бібліотека *pandas* використовується для роботи з табличними даними та зчитування CSV-файлу. Модуль *re* застосовується для очищення тексту за допомогою регулярних виразів. Бібліотеки з пакету *scikit-learn* використовуються для розбиття даних, перетворення тексту у числовий вигляд та навчання моделі машинного навчання. Модуль *pickle* необхідний для збереження навченої моделі та векторизатора у файли.

Далі у програмі виконується завантаження датасету з файлу emails.csv. Дані зчитуються у таблицю, після чого для зручності роботи перейменовуються стовпці: текст електронного листа зберігається у стовпці *message*, а відповідна мітка класу – у стовпці *label*.

```
# 1. Завантаження датасету
df = pd.read_csv("emails.csv")
df.columns = ["message", "label"]
```

Після завантаження даних виконується попередня обробка тексту. Для цього створюється окрема функція, яка приводить текст до нижнього регістру, видаляє зайві символи, посилання та електронні адреси, а також прибирає надлишкові пробіли. Така обробка необхідна для того, щоб зменшити кількість шуму у текстових даних та зробити їх більш придатними для навчання моделі.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", " url ", text)
```

```
    text = re.sub(r"\S+@\S+", " email ", text)
    text = re.sub(r"[^a-zA-Z ]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["message"] = df["message"].apply(clean_text)
```

Наступним кроком є розбиття датасету на навчальну та тестову вибірки. Для цього використовується стандартний підхід, за яким 80% даних використовуються для навчання моделі, а 20% – для перевірки її якості. Такий поділ дозволяє оцінити, наскільки добре модель працює на нових, раніше не бачених даних.

```
# 2. Розбиття на train/test
X_train, X_test, y_train, y_test = train_test_split(
    df["message"], df["label"], test_size=0.2, random_state=42
)
```

Оскільки модель машинного навчання не може працювати безпосередньо з текстом, на наступному етапі виконується векторизація текстових даних. Для цього застосовується метод TF-IDF, який перетворює кожен текст у числовий вектор. Даний метод дозволяє врахувати важливість слів у тексті та зменшити вплив часто вживаних, але малозначущих слів.

```
# 3. Векторизація тексту
vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1,2), min_df=5)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

Після векторизації тексту здійснюється навчання моделі класифікації. У даній лабораторній роботі для цього використовується алгоритм логістичної регресії, який добре підходить для задач бінарної класифікації та є простим для реалізації. Модель навчається на навчальній вибірці та після цього перевіряється на тестових даних, про що виводиться значення точності класифікації.

```
# 4. Навчання моделі
model = LogisticRegression()
model.fit(X_train_vec, y_train)

print("Accuracy:", model.score(X_test_vec, y_test))
```

Завершальним етапом навчання є збереження навченої моделі та векторизатора у файли ***model.pkl*** та ***vectorizer.pkl***. Це дозволяє надалі використовувати модель у веб-додатку без необхідності повторного навчання, що значно спрощує та прискорює роботу програми.

```
# 5. Збереження
pickle.dump(model, open("model.pkl", "wb"))
pickle.dump(vectorizer, open("vectorizer.pkl", "wb"))

print("Модель і векторизатор збережено!")
```

Після написання коду файл ***train_model.py*** запускається на виконання. У процесі виконання програми в консолі відображається інформація про результат навчання моделі. Зокрема, виводиться значення точності (accuracy), яке у даному випадку становить близько 97%. Також у консолі з'являється повідомлення про успішне збереження моделі та векторизатора.

```
Accuracy: 0.9738219895287958
Модель і векторизатор збережено!
```

Рисунок 3 — Результат виконання файлу train_model.py

Після завершення виконання програми у папці проєкту з'являються нові файли – ***model.pkl*** та ***vectorizer.pkl***. Ці файли містять відповідно навчену модель машинного навчання та об'єкт векторизації тексту. Саме вони надалі використовуються у веб-додатку для виконання класифікації введених користувачем повідомлень без необхідності повторного навчання моделі.

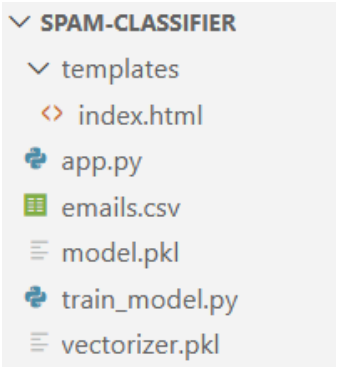


Рисунок 4 — Структура папки проєкту після навчання моделі

Використання навченої моделі у веб-додатку

Після завершення етапу навчання моделі наступним кроком є її використання у веб-додатку. Для цього було створено файл ***app.py***, який відповідає за роботу серверної частини програми та взаємодію з користувачем через браузер.

У даному файлі використовується веб-фреймворк Flask, який дозволяє швидко створювати прості веб-додатки на мові Python.

На початку файлу відбувається підключення необхідних бібліотек, зокрема ***Flask*** для роботи з веб-запитами та бібліотеки ***pickle*** для завантаження раніше збереженої моделі машинного навчання і векторизатора.

```
from flask import Flask, render_template, request
import pickle
```

Далі у програмі створюється об'єкт веб-додатку, після чого з файлів ***model.pkl*** та ***vectorizer.pkl*** завантажуються навчена модель та TF-IDF

векторизатор. Це дозволяє використовувати результати навчання без необхідності повторного запуску процесу навчання моделі.

```
app = Flask(__name__)

model = pickle.load(open("model.pkl", "rb"))
vectorizer = pickle.load(open("vectorizer.pkl", "rb"))
```

У веб-додатку реалізовано два основні маршрути. Перший маршрут відповідає за відкриття головної сторінки та відображення HTML-шаблону з формою для введення тексту.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Другий маршрут обробляє дані, які вводить користувач. Після натискання кнопки введений текст передається з веб-сторінки у програму, де він перетворюється у числовий вектор за допомогою векторизатора, а потім передається у модель машинного навчання для класифікації.

```
@app.route("/predict", methods=["POST"])
def predict():
    text = request.form["message"]
    vector = vectorizer.transform([text])
    prediction = model.predict(vector)[0]

    result = "SPAM" if prediction == 1 else "NOT SPAM"

    return render_template("index.html", message=text, result=result)
```

На основі результату роботи моделі формується відповідь, яка визначає, чи є введене повідомлення спамом або звичайним електронним листом. Отриманий результат передається назад у HTML-шаблон та відображається користувачу у браузері у зрозумілому вигляді.

Для запуску веб-додатку використовується стандартний механізм запуску Flask-додатків. Після запуску програми користувач може відкрити веб-сторінку у браузері, ввести текст електронного листа та отримати результат його класифікації.

Для взаємодії з користувачем у веб-додатку використовується HTML-шаблон ***index.html***, який розміщений у папці ***templates***. Даний файл містить форму для введення тексту електронного листа, кнопку для запуску перевірки та блок для відображення результату класифікації. Шаблон отримує дані з Flask-додатку та відображає результат у зручному для користувача вигляді.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```

```
<title>Email Spam Classifier</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background: #f5f7fa;
    margin: 0;
    padding: 40px 0;
    display: flex;
    justify-content: center;
  }
  .container {
    width: 100%;
    max-width: 520px;
    background: #fff;
    padding: 25px;
    border-radius: 10px;
    border: 1px solid #e0e0e0;
  }
  h2 {
    text-align: center;
    margin-bottom: 20px;
    color: #333;
    font-size: 24px;
  }
  textarea {
    width: 100%;
    height: 120px;
    font-size: 15px;
    border: 1px solid #ccc;
    border-radius: 6px;
    resize: none;
  }
  button {
    width: 100%;
    margin-top: 15px;
    padding: 12px;
    font-size: 16px;
    background: #4a90e2;
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
  }
  button:hover {
    background: #3b78c2;
  }
  .result {
    margin-top: 15px;
    padding: 12px;
    border-radius: 6px;
    text-align: center;
    font-size: 18px;
    font-weight: bold;
  }
  .spam {
    background: #fdeaea;
    color: #c62828;
  }
  .ham {
```

```
        background: #e8f5e9;
        color: #2e7d32;
    }
</style>
</head>
<body>

<div class="container">
    <h2>Email Spam Classifier</h2>
    <form action="/predict" method="POST">
        <textarea name="message" placeholder="Enter text to classify...">{{ message }}</textarea>
        <button type="submit">Check</button>
    </form>
    {% if result %}
        <div class="result {% if result == 'SPAM' %}spam{% else %}ham{% endif %}">
            {{ result }}
        </div>
    {% endif %}
</div>
</body>
</html>
```

Після запуску веб-додатку користувачу відображається сторінка з текстовим полем для введення повідомлення та кнопкою перевірки. Результат класифікації відображається на цій же сторінці у вигляді повідомлення про те, чи є введений текст спамом або звичайним електронним листом.

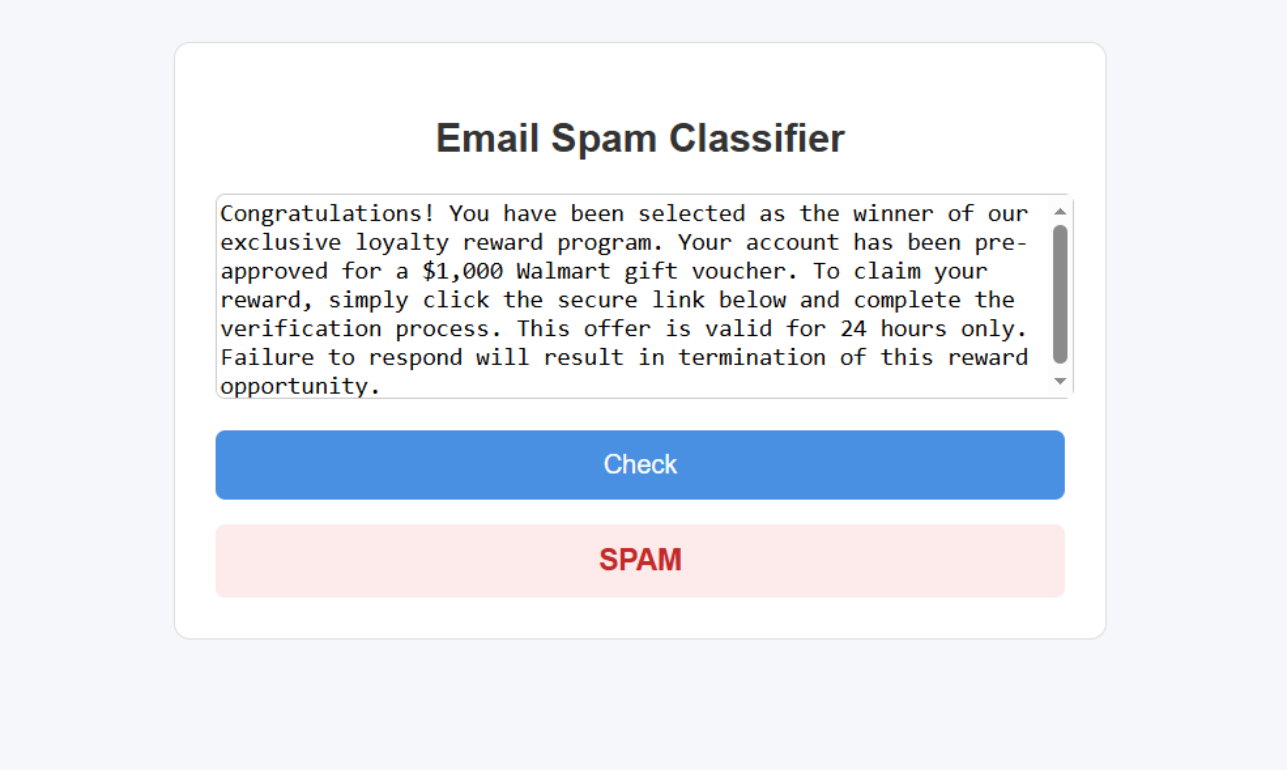


Рисунок 5 — Вигляд веб-додатку у браузері

Завдання на самостійну роботу

З метою закріплення матеріалу, отриманого під час виконання лабораторної роботи, студентам пропонується самостійно реалізувати

аналогічний веб-додаток з використанням моделі машинного навчання, але на основі іншого текстового датасету.

Варіант 1 — Класифікація тексту AI vs Human

Датасет: https://www.kaggle.com/datasets/shanegerami/ai-vs-human-text

Завдання: Реалізувати веб-додаток, який класифікує текст написаний людиною або штучним інтелектом.

Варіант 2 — Sentiment Analysis на IMDB Movie Reviews

Датасет:https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

Завдання: Реалізувати систему класифікації оглядів фільмів на позитивні / негативні.

Варіант 3 — Класифікація Twitter Disaster Tweets

Датасет: https://www.kaggle.com/datasets/vstepanenko/disaster-tweets

Завдання: Реалізувати веб-додаток, який визначає чи твіт пов’язаний з катастрофою чи ні.

Варіант 4 — Clickbait Detection

Датасет: https://www.kaggle.com/datasets/amananandrai/clickbait-dataset

Завдання: Реалізувати систему, яка класифікує заголовки / тексти на clickbait та не clickbait.

Варіант 5 — Text Emotion Recognition

Датасет:https://www.kaggle.com/datasets/shreejitcheela/text-emotion-recognition?select=train.csv

Завдання: Реалізувати веб-додаток, який класифікує текст за емоціями (sad, happy).